Your First Python Programs

Engineering Computation with Python — Chapter 2

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1. Introduction
2. Data Types
3. Editors and IDEs
4. Summary

Table of Contents

1 Introduction

2 Data Types

3 Editors and IDEs

4 Summary

Chapter 1 described what programs are and how algorithms are expressed on paper. This chapter is where that becomes real.

What you will do this week

Set up the software, type your first Python statements, and watch the machine execute them.

Why it matters

The ideas here — variables, types, the editor you work in — are small individually, but everything else in this course is written on top of them.

© Muním Zabidi

A Programming Language Is Like a Natural Language

Learning Python can be approached the same way you learned a natural language: built from a grammar and a dictionary.

Syntax	Variables	Operation
The layout of words and symbols. *Like:* grammar.	A stored value, retrieved by name. *Like:* a noun.	An action on variables. *Like:* a verb.

Syntax is the grammar; variables are the nouns; operations are the verbs.

© Munim Zabidi

Every Variable Carries Three Properties

A variable has

- **Data type** — what kind of value
- **Size** — how many elements (one, for now)
- **Value** — the actual content

An operation is expressed as

- **Statement** — a command
- **Function** — a group of statements
- **Type** — assignment, decision, repetition

© Muním Zabidi

Creating a Variable

A variable is created simply by assigning a value to it:

```
a = 5
```

Read as: *variable 'a' is assigned a value of 5.* Python stores the value in memory and names that location 'a'.

Using a name before it exists

```
a = 1 + b  # 'b' has never been assigned
```

```
NameError:  name 'b' is not defined
```

Dynamic Typing

Python automatically determines the type of a variable based on the value assigned to it, at runtime. This is called dynamic typing.

Contrast

You never declare “this variable will hold an integer” before using it, the way some other languages require. Python figures it out the moment a value is assigned.

The next section surveys the basic types a value can take.

© Muním Zabidi

Checking What Python Actually Did: `id()`

The built-in `id()` function returns a variable's memory address — where the value it points to actually lives.

```
x = 3
print(id(x))
x = 3 + 1
print(id(x))
```

Reassignment is not mutation

Integers are immutable: reassigning `x` does not change the object `3` — it makes `x` point to a *different* object, `4`, with a different `id()`. The original `3` is untouched.

Table of Contents

1 Introduction

2 Data Types

3 Editors and IDEs

4 Summary

Integer and Float

Integer (`int`) — whole numbers, positive or negative, no decimals.

```
x = 10
y = -5
print(type(x))
# <class 'int'>
```

Float (`float`) — numbers with a decimal point or scientific notation.

```
z = 3.14
w = 1.2e3  # 1.2 * 10^3
print(type(z))
# <class 'float'>
```

String and Boolean

String (`str`) — sequences of characters, used for text.

```
name = "Alice"
greeting = 'Hello!'
print(type(name))
# <class 'str'>
```

Boolean (`bool`) — `True` or `False`.

```
is_valid = True
has_perm = False
print(type(is_valid))
# <class 'bool'>
```

NoneType: The Absence of a Value

NoneType (`None`) represents the absence of a value — “nothing” or “no value here.”

```
value = None
print(type(value))
# <class 'NoneType'>
```

These five are scalar types

Each holds exactly one value at a time. Collections and complex numbers arrive in Chapter 3; the NumPy array — for holding many values at once — arrives in Chapter 6.

Three Mismatches That Trip Up Every Beginner

You already know arithmetic from mathematics courses. Python's numeric types let you do the same calculations — but Python's syntax is not identical to handwritten shorthand.

1. Implied multiplication does not exist.
2. The ^ symbol is not exponentiation.
3. Order of operations must be made explicit with parentheses.

Worth confronting immediately — these cause some of the most common silent errors in this course.

Mismatch 1: Implied Multiplication

In algebra, $2t$ or $3(x + 1)$ silently means “multiply.” Python has no such shorthand: $*$ must always be written.

```
t = 5

# result = 2t   # SyntaxError -- not valid Python

result = 2 * t  # Correct: 10
```

Rule

Every multiplication needs an explicit $*$, with no exceptions.

Mismatch 2: The Caret Is Not a Power

In mathematics, x^2 means "x squared." In Python, ^ is bitwise XOR — it has nothing to do with powers.

```
x = 5

wrong = x ^ 2   # Bitwise XOR. Output: 7
right = x ** 2  # Exponentiation. Output: 25
```

Why this trips people up

Excel and many calculators *do* use ^ for powers. Python does not. Always use **.

Mismatch 3: Parentheses Must Be Explicit

Translate the quadratic formula:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The fraction bar silently groups the numerator and denominator — both groupings must become explicit parentheses:

```
import math
a, b, c = 1, -3, 2

x1 = (-b + math.sqrt(b**2 - 4*a*c)) / (2*a)
x2 = (-b - math.sqrt(b**2 - 4*a*c)) / (2*a)
# Output: 2.0 1.0
```

© Muním Zabidi

The Danger of a Missing Parenthesis

Leaving out the parentheses around `2*a` silently computes a completely different (and wrong) expression:

```
# WRONG: divides only the sqrt term by 2, then
# multiplies by a
x1_wrong = -b + math.sqrt(b**2 - 4*a*c) / 2*a
```

The lesson

This looks almost identical to the correct version on the page — but computes something entirely different. Always check every grouping the formula implies.

Worked Example: Compound Interest

$$A = P\left(1 + \frac{r}{n}\right)^{nt}$$

```
P = 1000   # Principal
r = 0.05   # Annual interest rate (5%)
n = 12     # Compounded monthly
t = 10     # 10 years

A = P * (1 + r/n)**(n*t)
print(f"A = {A:.2f}")  # A = 1647.01
```

Every implied grouping becomes an explicit parenthesis: the whole `(1 + r/n)` is raised to the power `(n*t)` — not just to `n`.

Angles Are Measured in Radians, Not Degrees

Python's trig functions in `math` expect radians. `math.sin(30)` does *not* compute $\sin 30^\circ$.

```
import math

print(math.sin(30))               # -0.988  (30 RADIANS)
print(math.sin(math.radians(30))) #  0.5    (30 degrees)
print(math.degrees(math.pi / 4))  #  45.0   (back to degrees)
```

No error is raised

30 radians is a perfectly legal angle — Python computes a silently wrong answer rather than crashing. This is one of the most frequent sources of silently wrong engineering results.

Quick Reference: Math Notation to Python

Math	Wrong Python	Correct Python
$2t$	`2t` (SyntaxError)	`2 * t`
x^2	`x ^ 2` (wrong)	`x ** 2`
$\frac{a+b}{c+d}$	`a + b / c + d`	`(a+b)/(c+d)`
$\sin 30^\circ$	`math.sin(30)`	`math.sin(` `math.radians(30))`

Converting Between Types

Python provides built-in functions to convert one type to another:

```
a = int(3.99); b = int('20'); c = int(True)
# Outputs: 3, 20, 1

d = float(10); e = float('3.14'); f = float(False)
# Outputs: 10.0, 3.14, 0.0

g = str(123); h = str(3.14); i = str(True)
# Outputs: '123', '3.14', 'True'

j = bool(1); k = bool(0); l = bool('Hello')
# Outputs: True, False, True
```

The One Exception: NoneType

Unlike the other types, `NoneType` cannot be produced by converting a value with `int()`, `float()`, `str()`, or `bool()`.

```
value = None  # the ONLY way to obtain None
print(type(value))  # <class 'NoneType'>
```

Remember

There is no `none()` function. `None` must be assigned directly.

Operations That Produce NaN

Certain arithmetic operations produce the special floating-point value `NaN` (Not a Number):

- Any arithmetic operation *on* `NaN`
- Adding positive infinity to negative infinity
- Multiplying zero by infinity
- Dividing zero by zero, or infinity by infinity
- Remainder (modulo) with zero or infinity

These are legal, not errors

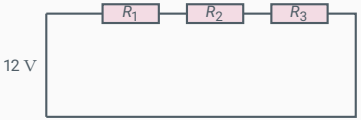
Python computes `NaN` without complaint — which is exactly why it is dangerous in a measurement. Chapter 11 returns to this: a `NaN` marks a reading that is missing.

Putting It Together: Resistors in Series

Components in series sit end to end on a single path, so the *same current* flows through all of them.

$$R_{\text{series}} = R_1 + R_2 + R_3.$$

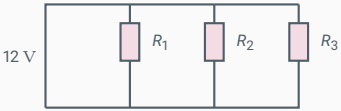
$$I = \frac{V}{R_{\text{series}}}, \quad V_k = IR_k.$$



Components in parallel are connected across the same two points, so each sees the *same voltage*.

$$\frac{1}{R_{\text{parallel}}} = \frac{1}{R_1} + \frac{1}{R_2} + \frac{1}{R_3}.$$

$$I_k = \frac{V}{R_k}.$$



© Munim Zabidi

The Reason `input()` Belongs Here

Rather than typing component values into the program, we can ask for them with `input()`.

The essential point

`input()` always returns a **string**, no matter what the user types.

```
R = input("R1 (ohms): ")  # R is "100", not 100
R = R * 2                 # '100100' -- repetition, not math!

R = float(input("R1 (ohms): "))  # now R is 100.0
```

Convert once, at the point of entry, and every later line can assume it has a number.

Reading the Circuit Values

```
V  = float(input("Supply voltage V (volts): "))
R1 = float(input("R1 (ohms): "))
R2 = float(input("R2 (ohms): "))
R3 = float(input("R3 (ohms): "))

R_series = R1 + R2 + R3
I_series = V / R_series

R_parallel = 1 / (1/R1 + 1/R2 + 1/R3)
```

Same three variables, two completely different formulas — the data type conversion pattern is identical either way.

© Muním Zabidi

Results: Series vs. Parallel

With 12 V, $R_1 = 100\,\Omega$, $R_2 = 220\,\Omega$, $R_3 = 330\,\Omega$:

```
--- Series ---
Total R:   650.0 ohms
Current:   18.462 mA
V(R1):     1.846 V
V(R2):     4.062 V
V(R3):     6.092 V
```

```
--- Parallel ---
Total R:   56.90 ohms
Total I:   210.909 mA
I(R1):     120.000 mA
I(R2):     54.545 mA
I(R3):     36.364 mA
```

© Muním Zabidi

Check: do the numbers add up?

Series. Voltage drops sum to the supply: $1.846 + 4.062 + 6.092 = 12.000$ V. ✓

Parallel. Branch currents sum to the total: $120.000 + 54.545 + 36.364 = 210.909$ mA. ✓

Interpret: the surprising result

In series, total resistance (650 Ω) is larger than any one resistor. In parallel, total resistance (56.9 Ω) is smaller than the smallest resistor — a useful sanity check for every parallel calculation.

© Munim Zabidi

When the User Types Something Unusable

`float()` raises an error if the text is not a number:

```
V = float(input("Supply voltage V (volts): "))
# If the user types "12v" instead of "12":
# ValueError: could not convert string to float: '12v'
```

This is the right behaviour

The program stops rather than continuing with a meaningless value. An unnoticed bad input is far worse than a halted program. Chapter 11 shows how to catch such errors and re-prompt instead of crashing.

Table of Contents

1 Introduction

2 Data Types

3 Editors and IDEs

4 Summary

Code Editors vs. IDEs

Writing Python requires a text-editing tool. These fall into two categories:

Code editors

VS Code, Sublime Text, Vim, GNU Emacs, Atom, Notepad++.

IDEs

PyCharm, Spyder, Jupyter, IntelliJ IDEA.

What an IDE adds

It bundles essential tools — editor, debugger, console, variable inspector — into one interface, streamlining writing, testing, and debugging.

Why Spyder for This Course

Spyder is particularly well suited to scientific and numerical Python work:

- Integrated scientific tools (NumPy, SciPy)
- A variable explorer for easy data handling
- An IPython console for interactive coding
- Built-in plotting with Matplotlib
- Code completion, syntax highlighting, customizable layouts

The Three Panes of Spyder

Editor	Workspace	Console
Where you write and edit your Python scripts.	Variable Explorer — an overview of variables and objects currently in memory.	Commands entered line by line, for testing snippets interactively.

You can see your code, run it, and inspect the resulting values — all at once.

© Munim Zabidi

The `whos` Command

In Spyder's IPython Console, typing `whos` lists every current variable, with its type, size, and value:

```
whos
```

Why this matters

It is the fastest way to check what Python actually thinks a variable's type and value are — especially useful when a calculation gives an unexpected result.

Table of Contents

1 Introduction

2 Data Types

3 Editors and IDEs

4 **Summary**

What You Should Take Away

- A language has **syntax** (grammar), **variables** (nouns), and **operations** (verbs).
- Python uses **dynamic typing** — it infers a variable's type at runtime from the assigned value.
- Five **scalar** types: `int`, `float`, `str`, `bool`, `None`.
- Convert explicitly with `int()`, `float()`, `str()`, `bool()` — except `None`, which can only be assigned directly.

What You Should Take Away (continued)

- Translating formulas needs care: write `*` explicitly, use `**` for powers (never `^`), parenthesize every implied grouping, and convert degrees to radians before calling `math`'s trig functions.
- `input()` always returns a string — wrap it in `float()` or `int()` before using it in arithmetic.
- Series resistors add; parallel resistors' reciprocals add, giving a total *smaller* than the smallest resistor.
- An **IDE** like Spyder combines an editor, console, and variable explorer in one place.

This chapter built the foundation: variables, scalar types, conversion, and the editor you will work in.

- Chapter 3 builds on these basic types: complex numbers, strings, and Python's collection types, together with the engineering units a measured value carries.
- Every later program is still just variables and operations — now with richer types to work with.

Next: Numbers, Variables and Units

© Munim Zabidi